Financial Literacy for Students

Porter Clevidence
Antonio Duran-Ramos
Parth Gajjar
Arpit Vora

May 6, 2025

List of Contents

List of Figures

List of Tables

Project Plan

Introduction

This project aims to build a search engine that allows students to retrieve relevant documents from a financial dataset. It is part of an educational initiative to promote financial literacy. This enables students to better understand and handle real-world financial situations encountered during their professional lives. To achieve this goal we have designed and implemented an *Information Retrieval (IR)* system that processes textual data, builds an *index* over a document collection, and *ranks* documents in response to user queries. [1]

Team Roles and Responsibilities

Porter Clevidence

- Implemented BIM search
- Implemented BM25 search
- Implemented Language Model retrieval
- Implemented VSM search
- Search algorithm selection

Antonio Duran-Ramos

- Development environment setup
- Graphical interface design
- Written report [2]
- Figures and Tables
- Dataset statistics

Parth Gajjar

- Presented

Arpit Vora

- Slideshow presentation

Methods and Techniques

To ensure cross-platform reproducibility across all of our team members, we used `Docker` [8] for containerization. Docker allowed us to standardize our development environment so that our search engine behaved identical, regardless of the host machine's underlying operating system.

Within this Docker container, our project formalizes its Python dependencies with `uv` [9] as the the primary package and virtual environmental manager. uv ensured exact reproducibility of the software state for implementation of the search engine.

Within uv, the core retrieval logic and presentation layers are constructed with various Python libraries:

- `pandas` was used to parse the provided JSON dataset files to create structured data frames
- `nltk` and `scikit-learn` were used for natural language processing tasks; `nltk` for its implementation of the *Porter Stemming* algorithm and `scikit-learn` for its list of English stop words
- `scikit-learn` was also used for its functions `TfidfVectorizer` and `cosine_similarity`
- `rank-bm25` was used for its implementation of the BM25 ranking algorithm
- `streamlit` [3] was used to implement the graphical interface of the search engine
- `matplotlib` was used to render figures and graphs based on retrieved data

Data

The dataset provided consisted of a collection of financial documents, along with static queries and their corresponding relevance judgments.

Data Description and Analysis

Dataset Statistics

The dataset consists of a collection of $5,000$ financial text documents, 25 static queries, and corresponding relevance judgments. This dataset was provided in separate JSON files.

From these 5000 documents, after tokenization, the average document length is 63.61 tokens. From the 25 queries, after tokenization, the average query length is 6.08 tokens.

There are 243 total relevant judgments. Additionally, there are an average of 9.72 relevant docs per query, of which the maximum is 15 and the minimum is 7.

Dataset Description and Analysis

Metric	Value
Total Documents	5,000
Total Queries	25
Average Document Length	63.61 tokens
Average Query Length	6.08 tokens
Total Relevant Judgments	243
Avg Relevant Docs per Query	9.72
Max Relevant Docs for a Query	15
Min Relevant Docs for a Query	7

Table 1: Provided dataset statistics evaluated after processing with `pandas` and our `DataProcessing` class.

Text Processing

Surface Level Normalization Definition

Before tokenizing and any further text processing, the text undergoes a three-stage cleaning process to ensure variations in formatting do not create duplicate tokens in our index.

1. The raw text is cast to lowercase with the `.lower()` function for strict case-insensitivity.
2. We strip out any punctuation, including periods, commas, quotes, brackets, and special symbols using a regular expression. This was applied with `re.sub(r’[^\w\s]’, ")`, which identifies any character that is *not* a word character (numbers included) or a whitespace character, and replaces it with nothing.
3. We again use a regular expression to target contiguous sequences of whitespaces, collapsing them to a single, clean space. Additionally, we apply the `.strip()` function to this regular expression to trim any lingering spaces at the beginning and end of a document.

Tokenizer Definition

The now surface-level normalized text is segmented into discrete vocabulary tokens by calling the `.split()` function along the whitespace boundaries.

Stopping Definition

Simultaneously alongside the tokenization, each token is immediately checked against the `ENGLISH_STOP_WORDS` list provided by the `scikit-learn` library. If the token is a highly common word, (i.e., "the", "is", "at", etc.), the token is instantly discarded. This results in a final tokens list full of meaningful words.

Stemming/Lemmatizing Definition

The remaining list of tokens is then looped over by `nltk`’s `PorterStemmer()` for algorithmic suffix-stripping. This algorithm removes common English suffixes to ensure that our vocabulary word variants converge to a single root index term.

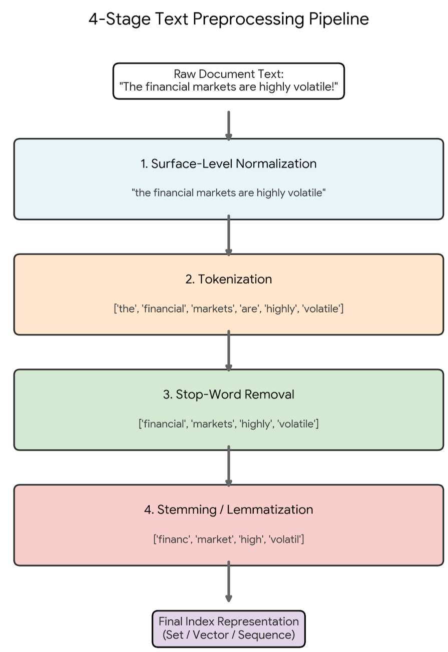

Figure 1: An example string passed through the four-stage text processing pipeline. Raw text is passed through surface-level normalization, tokenization, stop-word removal, and stemming before use in the system’s index.

Index Construction

Term Weighting Definition

Our codebase evaluates four distinct term weighting methodologies with third-party libraries and custom mathematical implementations.

Term frequency and inverse document frequency (TF-IDF) are calculated globally during the instantiation of the `DataProcessing` class. `TfidfVectorizer` normalizes the document vectors to a uniform length, regardless of the original document's size. Then, at query time, the `VSMSearch` class applies the `vectorizer.transform()` function to the query string. This function call converts the query text into a numerical matrix. We then compute the retrieval scores using the `cosine_similarity()` metric against the pre-computed, sparse matrix.

Our BM25 implementation delegates the complex mathematics to the `rank-bm25` library. We initialize the `BM25Okapi` class with an array of tokenized document arrays. Query execution is then handled natively by passing the tokenized query array into the `.get_scores()` function. This function call outputs an array of probabilistic scores that correspond to the exact original document indices.

The BIM algorithm utilizes our custom-built scoring function, `_bim_score()`, that evaluates term presence. The algorithm does this by mapping each document to a Python `set` for distinct vocabulary extraction. The weight of a query term is then calculated dynamically with the following formula:

$$\text{Score} = \sum_{t \in Q \cap D} \log\left(\frac{N - n_i + 0.5}{n_i + 0.5}\right),$$

where N is extracted from the length of the `doc_ids` and n_i is retrieved from our pre-computed document frequency dictionary.

Our unigram and bigram Language Models are implemented manually for strict control of probability interpolation. The functions `_unigram_score` and `_bigram_score` map the probabilities to log space, summing the results. This algorithm prevents zero-probabilities by linearly interpolating the local maximum likelihood estimate and the collection probability using a static smoothing parameter, λ. To calculate for the probability that a document D is relevant to a query term t, the following formula was implemented:

$$\mathcal{P}(t|D) = \lambda\left(\frac{tf_{t,d}}{L_d}\right) + (1-\lambda)\left(\frac{cf_t}{L_c}\right),$$

where λ is the smoothing parameter statically set to 0.3, $tf_{t,d}$ is the term frequency of a document, L_d is the length of a document, cf_t is the collection frequency (i.e., how many times does the specific query term appear across the entire database of documents), and L_c is the length of the collection (i.e., what is the total number of words in the entire database).

Index Design

The foundation for our retrieval system's index design begins with utilizing the `pandas` library to read from the provided JSON dataset collection. This transforms the dataset to data frame objects within our `DataProcessing` class. Relevance judgments are aggregated using vectorized operations to optimize the evaluation phase. In other words, this transformed the tabular data into an efficient dictionary of `set` objects which enable $\mathcal{O}(1)$ lookups for relevant document verification during precision calculations. Furthermore, our system builds separate indexes dictated by the specific requirements of the retrieval models.

For the Vector Space Model, the index is constructed as a *Compressed Sparse Row (CSR)* matrix using `TfidfVectorizer`. This variant of the index abstracts the vocabulary mapping, storing the mathematical weights in memory for optimized matrix dot-products.

For our models that require fine frequency calculations (e.g., BIM and Language Models), the index is designed using hash maps. Our system constructs unigram and bigram term-frequency mappings. This design similarly allows for $\mathcal{O}(1)$ retrieval of local documents lengths, collection occurrences, and document frequencies required for probabilistic scoring.

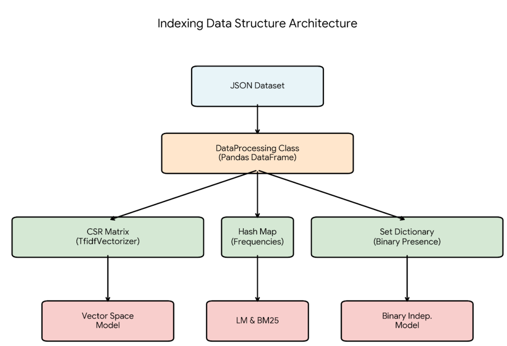

Figure 2: Index design process for the implemented retrieval models. All retrieval models share the same dataset.

Retrieval Model

Ranking

Before any relevance scores can be computed, the system projects the preprocessed queries and the document collection into mathematical representations. The mathematical representations are dictated by the requirements of the active retrieval model.

- For the Vector Space Model, both the documents and query are represented as high-dimensional, normalized vectors in a shared vocabulary space through the `TfidfVectorizor` function.
- For the BM25 and Language Models, documents and queries are represented as sequential lists of tokens. This list preserves the localized term frequencies and document lengths for probabilistic scoring and unigram or bigram sequences.
- For BIM, documents and queries are mapped to Python's `set` objects. We discard term frequencies and sequence data to evaluate binary term presence.

Then, during query execution, the search engine iterates over the document space and generates a relevance score for each document. Again, this score is dictated by the active retrieval model.

- For Vector Space Model, the system calculates the cosine angle between the query vector and document vectors. This yields a similarity score between 0.0 and 1.0. [5]
- For BM25, the system calculates a score based on term frequency and document length normalization constraints. [4][6][7]
- For BIM, the system iterates through the query terms, checking for set intersections with the document. This system dynamically sums the probabilistic weights for all intersecting terms. [5]
- For the Language Models, the system calculates the probability of the query given the document model. This system transforms the smooth probabilities into log space and sums them to prevent arithmetic underflow caused by multiplying small probabilities. [5]

After the relevance scores have been computed across the entire collection, the values are stored in a contiguous array, parallel to the original document ID index. To establish a final ranking order, the array is sorted from highest relevance score to lowest using Python's sorting capabilities. The ordered indices, along side their scores, are stored in a data frame that is then used for pagination in the graphical interface.

Model Representation and Scoring Matrix

Retrieval Model	Query/Doc Representation	Scoring Function	Complexity / Execution
VSM	Dense/Sparse Vectors (TfidfVectorizer)	Cosine Similarity Angle	Matrix Dot-Product
BM25	Token Sequences	Frequency & Length Normalization	Native .get_scores() Function
BIM	Python Sets	Binary Intersection & Probabilistic Sum	$\mathcal{O}(1)$ Hash Lookup
Language Models (Unigram/Bigram)	Token Sequences	Log-Space Probability Sum	Interpolated Background Smoothing

Table 2: Details of the query and document representations, underlying mathematical scoring functions, and computational complexities across the implemented retrieval algorithms.

Table 3: Visual representation of the smoothing balance, where $\lambda = 0.3$, illustrating the proportional weighting between local document likelihood and the global collection.

Implementation

Source Code

https://github.com/toniodr/Financial-Literacy-for-Students/tree/main

Video Demonstration

Discussion

This project provided extensive insight into the methods and concepts we learned throughout the course of this semester. The insight of this project came from not only implementing the retrieval models we learned, but also by analyzing and comparing the results of the individual retrieval models.

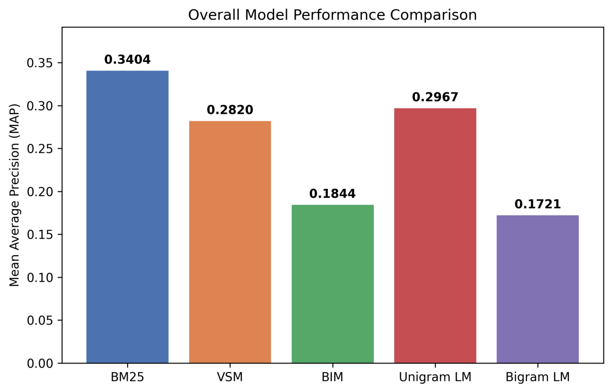

Figure 3: The Mean Average Precision (MAP) scores of the implemented retrieval models with BM25 having the highest score, followed by the Unigram Language Model, VSM, BIM, and finally the Bigram Language Model.

Across the 5,000 financial document corpus, retrieval model BM25 was the overall better performing model when compared to its counterparts.

Final Evaluation Metrics

Retrieval Model	MAP Score	P@5 Score
BM25	0.3404	0.4480
Unigram LM	0.2967	0.3840
VSM	0.2820	0.3840
BIM	0.1844	0.2240
Bigram LM	0.1721	0.2640

Table 4: MAP and P@5 scores for the implemented retrieval models ordered by MAP score.

As evidenced by *Figure 3* and *Table 4*, BM25 consistently achieved the highest Mean Average Precision (MAP) and Precision at 5 (P@5) scores. We attribute these findings to the algorithm's professionally implemented probabilistic framework. This framework balances term frequency saturation with document length normalization, an elegance that our custom built algorithms and mathematical computations may be lacking. Additionally, the document dataset consisted of various lengths, a property that BM25 mitigates by preventing longer, repetitive documents from artificially inflating their relevance scores.

However, despite the overall dominance of the BM25 algorithm, performance of this model severely degraded with the provided secret queries. We hypothesized that this algorithm would maintain high retrieval accuracy, but, the visual and numerical results of the blind evaluation revealed a performance degradation, invalidating our initial assumption.

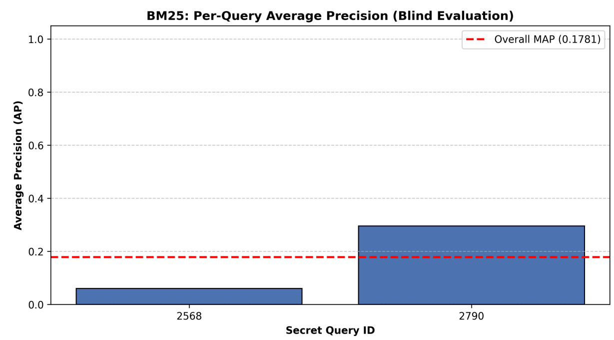

Figure 4: Average Precision (AP) of the BM25 retrieval model against the secret queries, with an overall MAP score of 0.1781.

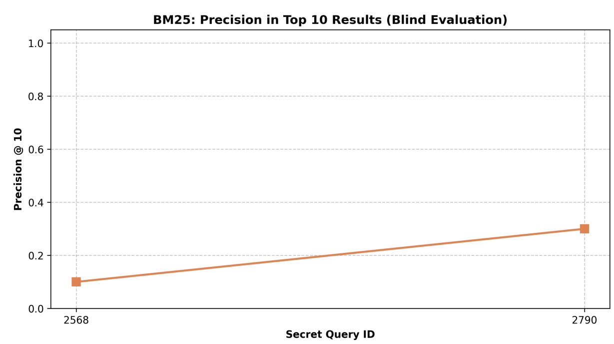

Figure 5: Precision at 10 scores for the BM25 retrieval model against the two secret queries.

This degradation is corroborated by `Figure 4` and `Figure 5`. `Figure 4` shows us that the secret queries dropped the overall MAP score of BM25 significantly. Furthermore, `Figure 5` shows us that this retrieval model failed to return a notable amount of relevant documents within the top ten results for the secret queries.

We conclude that this performance degradation may be a cause of the strict probabilistic formula that penalizes prevalent words. This algorithm may be calculating zero weight terms for words in the queries, resulting in non-relevant documents to be returned. [4] These findings tell us that future iterations must implement vocabulary smoothing to safely handle common query terms.

References

[1] CS5180 Professor Marin, *Course Project Requirements: Financial Literacy For Students*. Computer Science Department, 2026.

[2] CS5180 Professor Marin, *Project Report*. Computer Science Department, 2026.

[3] Streamlit Inc., *Streamlit Documentation*. [Online]. Available: `https://docs.streamlit.io/`

[4] S. Robertson and H. Zaragoza, *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in Information Retrieval, 2009.

[5] W. Bruce Croft, Donald Metzler, and Trevor Strohman, *Search Engines: Information Retrieval in Practice*. Addison Wesley, 2010

[6] Hugging Face, *BM25 for Python: Achieving high performance while simplifying dependencies with BM25S*. [Online]. Available: `https://huggingface.co/blog/xhluca/bm25s`

[7] PyPi, *Rank-BM25: A two line search engine*. [Online]. Available: `https://pypi.org/project/rank-bm25/`

[8] Docker, *Docker Compose*. [Online]. Available: `https://docs.docker.com/compose/`

[9] uv, *uv*. [Online]. Available: `https://docs.astral.sh/uv/`